

ERIK ERP Developer brief – Finance & Accounting module

Below is a single, consolidated, developer-ready breakdown of everything we've discussed about Finance & Accounting for the ERIK ERP. It's organized so engineers (backend, frontend, QA, devops) can pick up tasks quickly: features, data model, APIs, frontends (Python Streamlit + HTML/CSS/JS notes), business rules, integrations, compliance, testing, acceptance criteria, and a prioritized implementation plan.

1 — High-level scope & goals

- Full general-ledger (GL) + AP + AR + fixed assets + cash/bank + multi-currency + VAT/Tax + payroll integration (payroll already defined in ERP) + invoicing (Smart Invoice-compliant) + reporting, reconciliation and audit trail.
 - Support both **compact journal entry format** (amount once with both debit & credit accounts) and legacy line-by-line double-entry for compatibility.
 - Designed as microservice API (FastAPI) with Python frontend (Streamlit) and optional HTML/CSS/JS frontends. Database: PostgreSQL (recommended) with transactional integrity.
 - Audit-first: immutable audit trail, versioning of financial documents, user roles & permissions.
-

2 — Modules / features (top-level)

1. Chart of Accounts (COA) manager (multi-tenant, hierarchical)
2. Journal Entries (compact & legacy)
3. Ledger & Trial Balance
4. Accounts Receivable (customer invoices, credit notes, receipts)
5. Accounts Payable (supplier bills, credit notes, payments)
6. Bank & Cash (bank accounts, bank statements, reconciliation engine)
7. Fixed Assets (capex, depreciation schedules)
8. Multi-currency engine (rates, effective-dates, revaluation)
9. VAT / Sales Tax engine & VAT return reports
10. Invoicing (Smart Invoice compliance, PDF/UBL exports)
11. Payments (manual, bulk, payment runs; integrate with payment gateways)
12. Intercompany transactions & eliminations
13. Financial reporting (P&L, Balance Sheet, Cash Flow, Custom reports)
14. Period close / lock periods / opening balances

15. Audit trail, approvals workflow, & role-based access controls
 16. Integrations: Payroll (internal), Banking (import MT940/CSV), OCR/GRN (procurement), external tax gateways
 17. Month / Year-end processes and automated adjustments (e.g., accruals, prepayments)
 18. API for data export (CSV, XLSX, JSON) and secure webhooks
 19. Smart Invoice Compliance revisions (modules 17,18,19,40,53 flagged for revision)
 20. Notifications & scheduled jobs (exchange rate updates, recurring invoices)
-

3 — Core business rules & important specifics (from prior decisions)

- **Compact journal entry structure:** single amount field with debit and credit accounts in same entry. System must map this to double-entry postings internally for reporting and reconciliations.
 - **Payroll integration specifics:** Payroll components included in gross salary (housing, lunch, transport, bonuses, gratuity), employer and employee NAPSA & NHIMA contributions shown separately; NHIMA = **1% of basic salary** only (not gross).
 - **VAT / Smart Invoice:** invoices must include required Smart Invoice metadata (supplier/customer tax IDs, invoice number, issue date, tax amounts, currency, exchange rates, invoice line-level data) and support both PDF & UBL/JSON export.
 - **Immutable audit trail:** all financial docs (journals, invoices, payments) must have write-once records with separate corrective documents rather than editing in-place (store edits as new version records).
 - **Multi-currency:** clients choose a home currency; they can add foreign currencies with configurable rate sources and effective dates. Show rates as “home currency per foreign currency” OR “foreign currency per home”, configurable per client.
 - **Reconciliation rules:** payments and receipts must be matchable to invoices/bills; support partial matches, auto-suggest matches, manual adjustments, and unapplied receipts.
 - **Module revision list:** Modules 17, 18, 19, 40, 53 must be revised for Smart Invoice compliance (per prior notes). Include umbrella heading in those revisions.
-

4 — Data model (essential tables / entities)

(Use PostgreSQL; include UUID PKs; timestamps; tenant_id for multi-tenancy)

Primary tables (abbreviated)

- tenants (id, name, home_currency, settings_json, created_at)

- accounts (id, tenant_id, code, name, type [asset/liability/equity/rev/exp], parent_id, normal_balance, is_reconcilable, created_at)
- journal_entries (id, tenant_id, journal_number, date, description, currency, total_amount, exchange_rate, status, created_by, created_at)
- journal_lines (id, journal_entry_id, account_id, side [debit/credit], amount, currency_amount, narration)
- invoices (id, tenant_id, invoice_number, type [sales/purchase], status, date_issued, due_date, currency, total_amount, tax_amount, seller_id, buyer_id, smart_invoice_json, pdf_path, created_at)
- invoice_lines (id, invoice_id, description, account_id, qty, unit_price, line_total, tax_code)
- payments (id, tenant_id, payment_number, date, amount, currency, method, reference, bank_account_id, status)
- bank_accounts (id, tenant_id, bank_name, account_no, currency, ledger_account_id)
- bank_statements_imports (id, tenant_id, file_meta, imported_at)
- bank_reconciliations (id, tenant_id, bank_account_id, period_start, period_end, reconciled_at, status)
- fixed_assets (id, tenant_id, name, asset_account_id, accumulated_depr_account_id, cost, salvage, useful_life_months, depreciation_method, active)
- exchange_rates (id, tenant_id, from_currency, to_currency, rate, source, effective_date)
- taxes (id, tenant_id, name, rate, tax_type, account_id)
- approvals (id, resource_type, resource_id, approver_id, status, comments, created_at)
- audit_logs (id, user_id, tenant_id, action, resource_type, resource_id, payload_json, created_at)
- attachments (id, resource_type, resource_id, filename, path, uploaded_by)

Include indices on tenant_id, date fields, account codes.

5 — Compact Journal JSON example (single-amount format)

```
{
  "tenant_id": "uuid-tenant",
  "journal_number": "J2025-0001",
  "date": "2025-10-29",
  "currency": "ZMW",
  "total_amount": 1200.00,
  "exchange_rate": 1.0,
  "description": "Sale of goods to Acme",
  "entries": {
    "debits": [
      {"account_code": "1000-AR", "amount": 1200.00, "narration": "Invoice
2025/001"}
    ],
    "credits": [
```

```

        {"account_code": "4000-SALES", "amount": 1200.00, "narration": "Goods
sold"}
    ]
},
"metadata": {"origin": "invoicing", "source_id": "invoice-uuid"}
}

```

Backend must expand to two `journal_lines` rows (debit & credit) for the GL.

6 — API design (FastAPI) — essential endpoints

Use bearer token (JWT) + RBAC.

Chart of Accounts

- GET `/api/v1/accounts` — list (filter: `tenant_id`, `type`, `searchable`)
- POST `/api/v1/accounts` — create
- PUT `/api/v1/accounts/{id}` — update (log version)
- DELETE `/api/v1/accounts/{id}` — soft-delete (if no transactions)

Journals & Posting

- POST `/api/v1/journals` — create journal (accept compact JSON or legacy lines)
- GET `/api/v1/journals/{id}` — retrieve
- POST `/api/v1/journals/{id}/approve` — approval workflow
- GET `/api/v1/ledger` — ledger query (filters, paging)
- GET `/api/v1/trial-balance` — trial balance (period)

Invoices

- POST `/api/v1/invoices` — create invoice (Smart Invoice fields supported)
- GET `/api/v1/invoices/{id}` — retrieve (include pdf download)
- POST `/api/v1/invoices/{id}/send` — email or webhook send
- GET `/api/v1/invoices/export?format=ubl|pdf|json` — exports

Payments & Reconciliation

- POST `/api/v1/payments` — create payment
- POST `/api/v1/payments/match` — match payment to invoice(s)
- POST `/api/v1/bank-statements/import` — import CSV/MT940
- POST `/api/v1/reconciliations/{bank_account_id}` — run auto-reconcile

Fixed Assets

- `POST /api/v1/fixed-assets` — create
- `POST /api/v1/fixed-assets/{id}/depreciate` — run depreciation (or scheduled job)

Multi-Currency & Rates

- `GET /api/v1/exchange-rates?date=YYYY-MM-DD`
- `POST /api/v1/exchange-rates` — create rate

Reports

- `GET /api/v1/reports/pnl?start=&end=¤cy=`
- `GET /api/v1/reports/balance-sheet?date=¤cy=`
- `GET /api/v1/reports/cashflow?period=`

Admin / Settings

- `GET /api/v1/settings/finance`
- `POST /api/v1/settings/finance` — update preferences (compact entry enabled, currency display, rounding)

Webhooks

- `POST /api/v1/webhooks/payment-received` — internal/external events

7 — Frontend (Streamlit + HTML/CSS/JS notes)

Priority: Streamlit frontends required for modules 1–39 per project list; HTML versions later.

Streamlit (Python) general guidance

- Use componentized pages: COA manager, Journal Entry form, Invoice form, Bank Reconciliation, Reports.
- Journal Entry UI:
 - Compact entry mode toggle (single amount + debit/credit pickers).
 - Auto-balancing validation (client-side).
 - Attachments support (PDF invoices).
 - Preview GL postings before submit.

- Invoice form:
 - Line-level tax selection.
 - Smart Invoice metadata fields (TIN, VAT reg no, currency, exchange rate, invoice type).
 - Generate PDF preview (server-side) and UBL/JSON export button.
- Bank reconciliation:
 - Upload statement file, auto-suggest matches, drag & drop UI for manual match, chart showing unreconciled items.
- Permissions:
 - Show/hide approve buttons based on RBAC.
- Use responsive layout and tooltips for accounting terms.

HTML/CSS/JS (for later)

- Provide REST endpoints and JSON schema; use React + Tailwind (shipped earlier in canmore guidelines).
 - Ensure accessible forms with validation.
-

8 — Smart Invoice compliance & invoicing specifics

- Invoice data model must include:
 - supplier/buyer legal name, address, tax ID (TIN), VAT status
 - invoice number, issue_date, due_date, currency, line_items with tax breakdown, total_tax, net_amount, gross_amount
 - digital signature / hash (optional) and invoice UUID
 - attachments / supporting documents
 - Provide export to PDF and to interoperable machine-readable format (UBL or JSON-LD) as required by local tax authorities.
 - Revise modules 17,18,19,40,53 (Invoice generation, Payslip/Payroll interactions with invoices, GRN/Matching) to ensure Smart Invoice fields are present.
-

9 — Multi-currency & exchange rates rules

- Each tenant has home_currency.
- Journal/invoice currency can differ from home. Store currency and currency_amount on lines and an exchange_rate snapshot on the transaction.
- Exchange rates must be timestamped with effective_date.
- Revaluation process:

- run scheduled revaluation job for balance sheet fx items (AR/AP, bank balances).
 - Post FX gains/losses to designated accounts defined in tenant settings.
 - Allow manual override on per-transaction basis with `exchange_rate_reason` field.
-

10 — Tax & statutory (VAT) engine

- Tax rates stored in `taxes` table with mapping to account codes.
 - Line-level tax computation; support tax-inclusive and tax-exclusive pricing.
 - VAT return generator:
 - Summarize output VAT, input VAT, VAT payable/refundable for a period.
 - Export in CSV/XLSX and PDF.
 - Support zero-rated, exempt, and reverse-charge treatments.
-

11 — Bank reconciliation & transaction matching

- Import statements (CSV/MT940). Map imported statement lines to system payments/receipts.
 - Auto-match rules:
 - exact amount/date match
 - fuzzy match: amount within tolerance + reference substring
 - description/clustering / rule engine (user-defined)
 - Manual matching UI and adjustment journal generation for bank fees/charges.
 - Keep reconciliation sessions per period; reconciled_at timestamp.
-

12 — Fixed assets & depreciation

- Support straight-line and declining balance.
 - Generate monthly depreciation journals automatically.
 - Support disposal with gain/loss posting and asset write-off.
-

13 — Approval & workflow

- Multi-level approvals for high value journals, payment runs, supplier invoices.
 - Configurable thresholds per tenant.
 - Approvals create separate records; do not alter original transaction (versioning).
-

14 — Security, integrity & audit

- Role-based access control (Admin, Accountant, Approver, Viewer, Payroll-Integrator).
 - All mutating actions write to `audit_logs` with payload, user, timestamp.
 - Immutable ledgers: once journal posted and period closed, edits via reversal/correction journal only.
 - Data encryption at rest for attachments and sensitive fields.
 - Audit exports with tamper-evident checksums.
-

15 — Integration points

- Payroll module: push employer/employee statutory contributions and payroll journals; provide `POST /api/v1/integrations/payroll/journal` to accept payroll summary journal.
 - Banking: import statement formats; optional bank API connectors (Plaid-style) if available later.
 - Procurement/OCR: receive GRN & invoice matches from procurement OCR module (module 79/GRN & delivery matching).
 - Payment gateways: webhook endpoint for online payments to mark invoices paid.
 - Tax authority APIs (if available) for e-invoice submission.
-

16 — Reporting & exports

- Core: P&L, Balance Sheet, Trial Balance, Cash Flow (direct & indirect), AR/AP aging, VAT returns.
 - Support dimension-based reporting (cost centers, departments, projects).
 - Scheduled PDF/XLSX exports and email reports.
 - Provide reporting API with filtering, pagination, and caching for large date ranges.
-

17 — Testing, QA & acceptance criteria

- Unit tests for financial math (line-level tax, rounding), currency conversions, and depreciation.
 - Integration tests for posting journal → ledger → trial balance balancing.
 - E2E tests for invoice → payment matching → bank reconciliation.
 - Security tests: RBAC, SQL injection, XSS in attachments.
 - Performance tests: ensure trial balance queries < 1s for 100k entries (benchmark target — adjust to infra).
 - Acceptance criteria (sample):
 - Creating a compact journal posts correct debit & credit lines and trial balance remains balanced.
 - Invoice created has Smart Invoice JSON attached and PDF generated; export UBL validates against schema.
 - Importing bank statement matches 80% of transactions via auto-rules in common cases.
-

18 — Deployment & background jobs

- Background workers (Celery / RQ / Prefect) for:
 - scheduled exchange rate updates
 - depreciation job
 - recurring invoices
 - bank statement imports & reconciliation tasks
 - Containerized services (Docker Compose / Kubernetes). DB migrations handled with Alembic.
 - Backups & point-in-time recovery for DB.
-

19 — Suggested milestones & priority roadmap (example)

1. Core: COA, Journal POST, ledger, trial balance (compact+legacy) — **M1**
2. AR module + invoicing (basic PDF) + payments — **M2**
3. AP module + vendor bills + payment run — **M3**
4. Bank accounts & reconciliation engine (CSV import) — **M4**
5. Multi-currency engine & exchange rates — **M5**
6. VAT & tax engine + VAT return generator — **M6**
7. Fixed assets & depreciation — **M7**
8. Smart Invoice compliance rewrites for flagged modules (17,18,19,40,53) — **M8**
9. Reporting, exports, audit & approvals — **M9**
10. Integrations (Payroll, procurement OCR, payment gateways) — **M10**

Deliver in vertical slices (e.g., M1 → fully testable), not feature-by-feature.

20 — Migration & backwards compatibility notes

- Provide migration scripts to convert legacy journal lines into compact format or vice versa. Keep both models available in DB.
 - Importer utility for historical transactions with mapping tools (CSV template).
-

21 — Edge cases & gotchas (must handle)

- Partial payments, multiple currency partial payments with rate-deltas.
 - Rounding differences causing balancing issues — provide tolerance and auto-adjust rounding journal entries.
 - Reconciliation of refunds & credit notes.
 - Closed period attempts — disallow or route to adjustment journal with approval.
 - Deleting accounts with history — disallow; only soft-close.
-

22 — Example developer deliverables (per sprint)

- API spec (OpenAPI/Swagger) for all endpoints described.
 - DB schema / migration scripts (SQL + Alembic).
 - Streamlit UI components for: COA editor, Compact Journal form, Invoice form, Bank reconciliation UI, Standard reports.
 - Unit / integration test suite with sample seed data.
 - Postman collection and sample tenant dataset.
 - README with operational runbook (backup, migration, versioning).
 - Smart Invoice validator & UBL/JSON export templates.
-

23 — Acceptance checklist for release

- ☐ API endpoints documented in OpenAPI and tested.
 - ☐ Compact journal form posts and expands correctly into ledger.
 - ☐ Trial balance and P&L produce correct numbers vs sample dataset.
 - ☐ Invoice generation (PDF + Smart JSON) validated.
 - ☐ Bank import + auto-match demo working on sample MT940/CSV.
 - ☐ Payroll integration endpoint passes sample payroll journal and posts correctly (NAPSA/NHIMA split).
 - ☐ Audit trail present and immutable for posted transactions.
 - ☐ Security: RBAC, auth, encryption validated.
 - ☐ Performance baseline logged.
-

24 — Quick reference: sample SQL for compact journal expansion

```
-- pseudocode example:
WITH je as (
  INSERT INTO journal_entries (id, tenant_id, journal_number, date,
total_amount, currency)
  VALUES (...)
  RETURNING id
)
INSERT INTO journal_lines (journal_entry_id, account_id, side, amount)
SELECT je.id, accounts.id, 'debit', payload->...'amount' FROM ...;
-- then insert credit line
```

(Implementation should be wrapped in a DB transaction.)

25 — Next steps I recommend you action immediately

1. Generate a full OpenAPI spec from the above endpoints to distribute to frontend and QA teams.
2. Scaffold FastAPI project with DB models + Alembic migrations for key tables above.
3. Build the Streamlit compact journal form (M1 vertical slice) and add unit tests for balancing logic.
4. Create sample tenant dataset including multi-currency examples and payroll-linked journal entries to validate end-to-end flows.

5. Schedule a short design review specifically for Smart Invoice fields to confirm local tax authority expectations and final schema for UBL/JSON.